

Queue



What is a Queue?

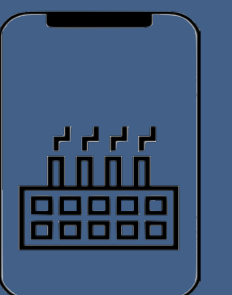
Queue is a data structure that stores items in a First-In/First-Out manner.



A new addition to this queue happens at the end of the queue.

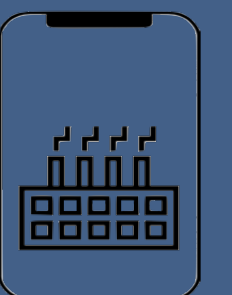
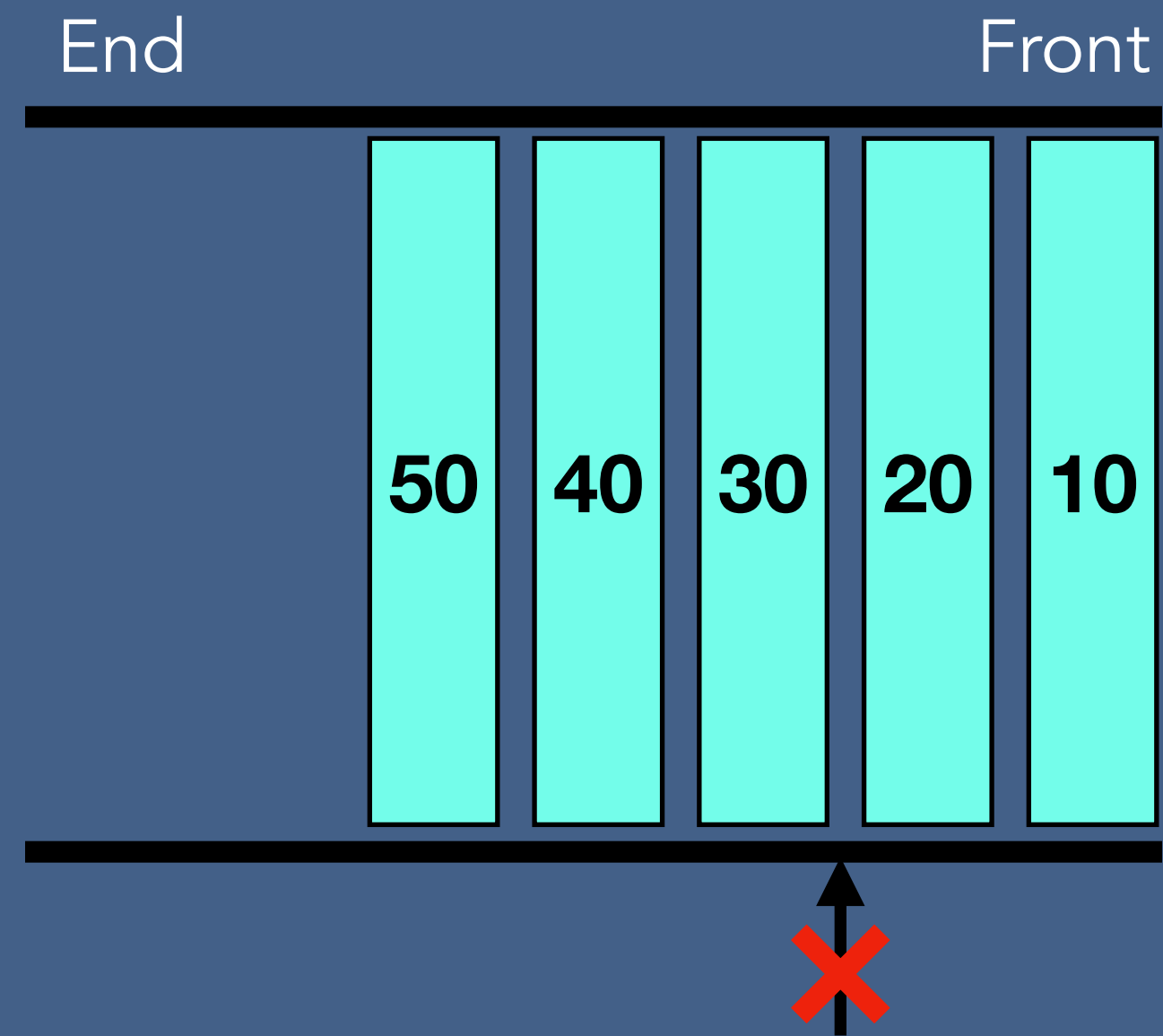
First person in the queue will be served first

FIFO method - First in First Out



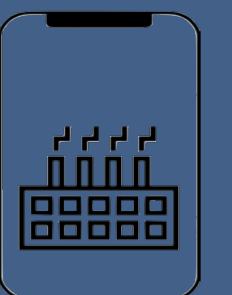
What is a Queue?

Queue is a data structure that stores items in a First-In/First-Out manner.



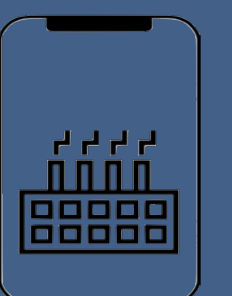
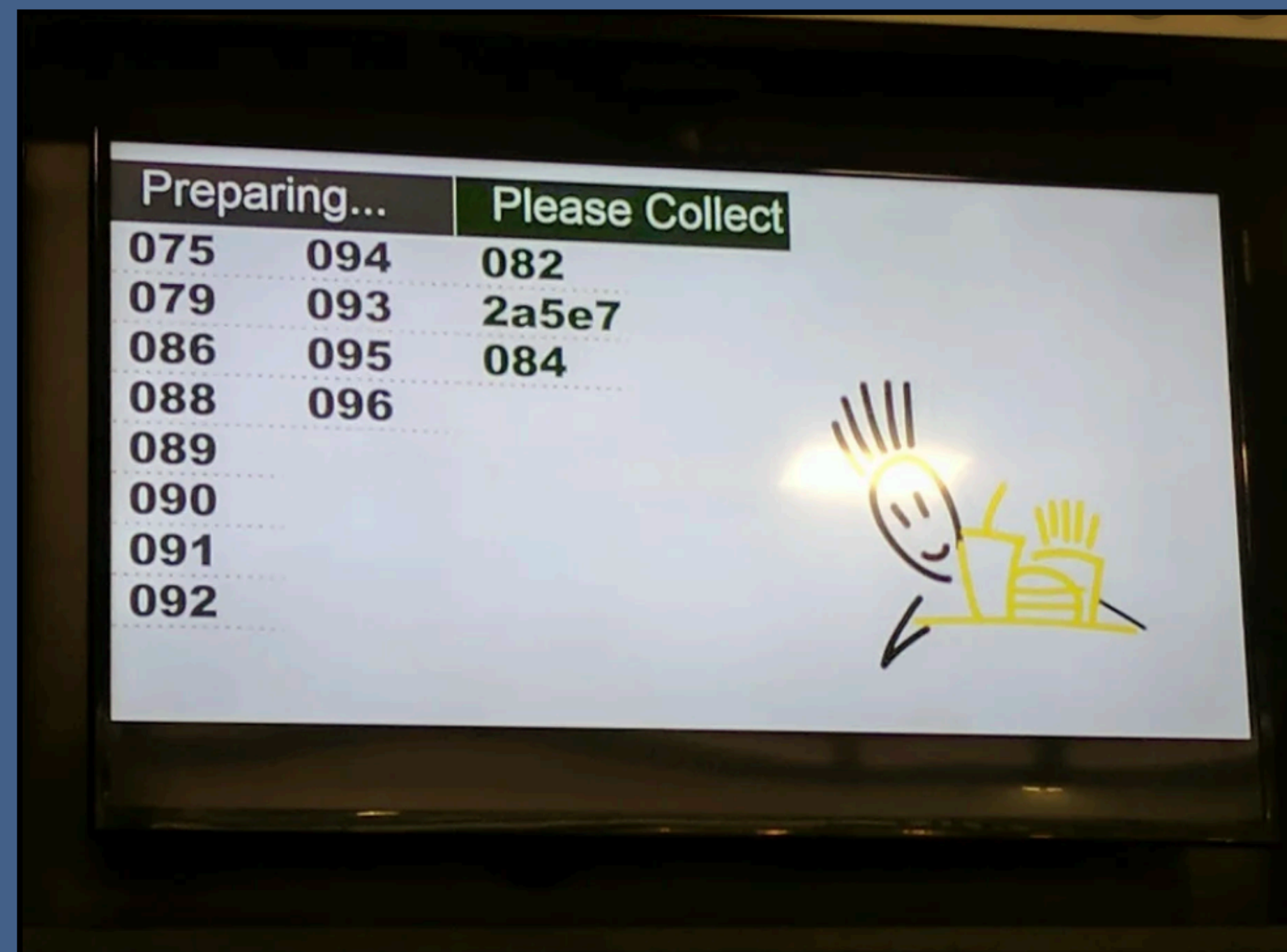
Why do we need Queue?

- Utilize first coming data first , while others wait for their turn.
- FIFO method - First In First Out



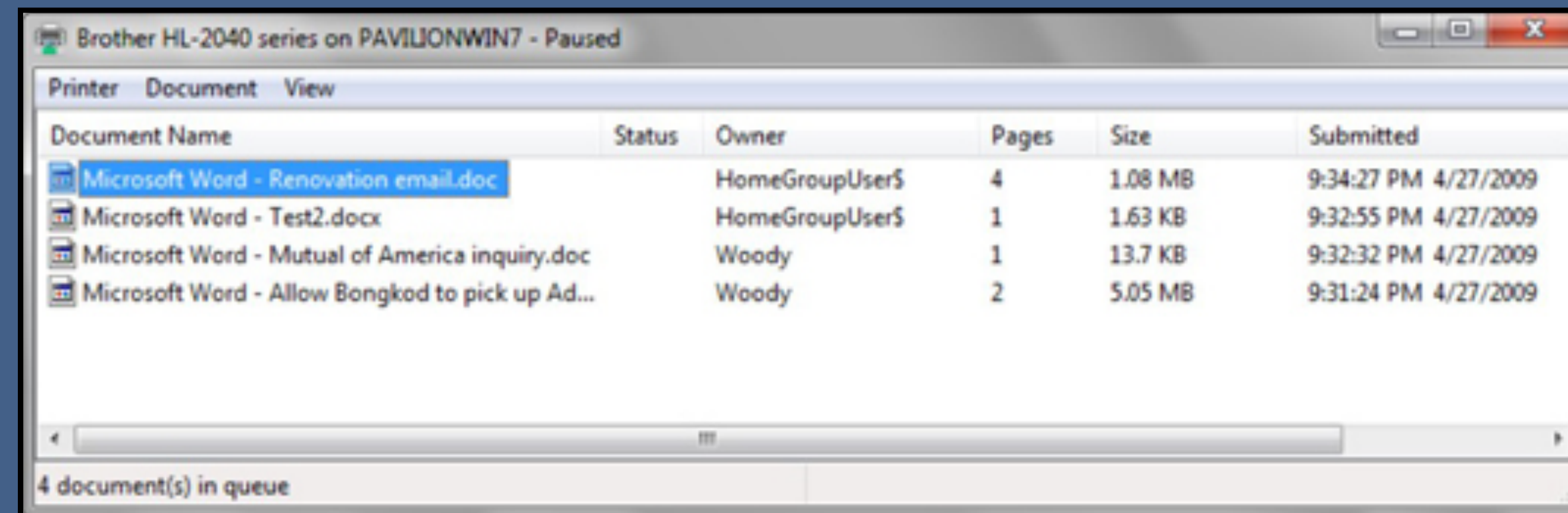
Why do we need Queue?

Point sale system of a restaurant



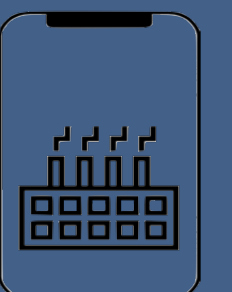
Why do we need Queue?

Printer queue



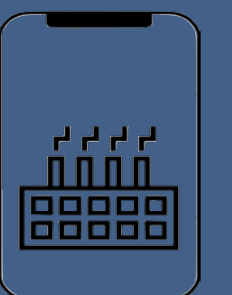
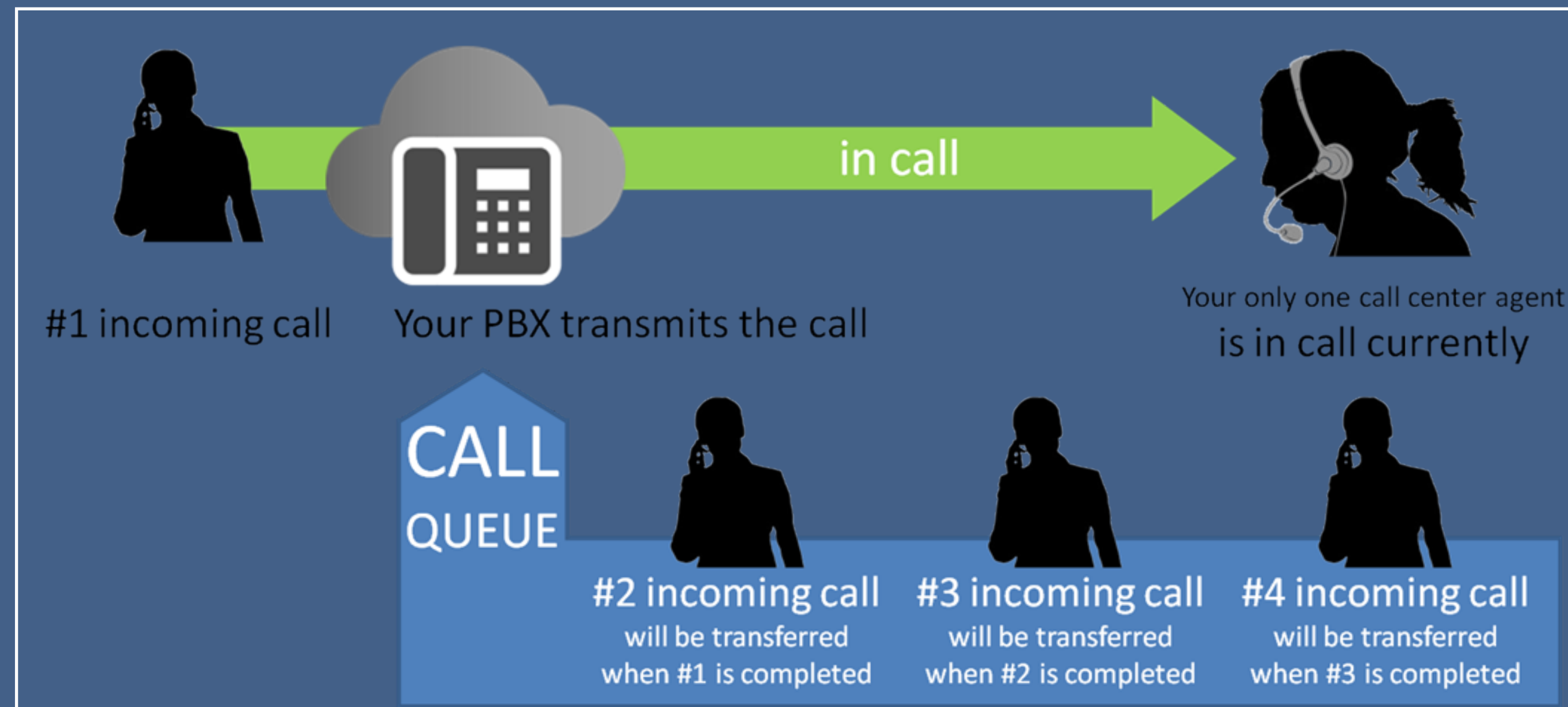
Printer Document View					
Document Name	Status	Owner	Pages	Size	Submitted
Microsoft Word - Renovation email.doc		HomeGroupUser\$	4	1.08 MB	9:34:27 PM 4/27/2009
Microsoft Word - Test2.docx		HomeGroupUser\$	1	1.63 KB	9:32:55 PM 4/27/2009
Microsoft Word - Mutual of America inquiry.doc		Woody	1	13.7 KB	9:32:32 PM 4/27/2009
Microsoft Word - Allow Bongkod to pick up Ad...		Woody	2	5.05 MB	9:31:24 PM 4/27/2009

4 document(s) in queue



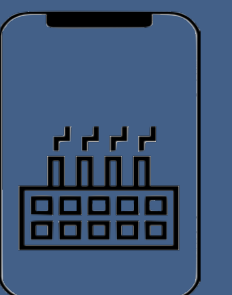
Why do we need Queue?

Call center phone systems



Queue Operations

- Create Queue
- Enqueue
- Dequeue
- Peek
- isEmpty
- isFull
- deleteQueue



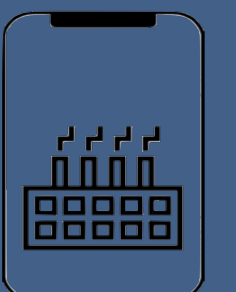
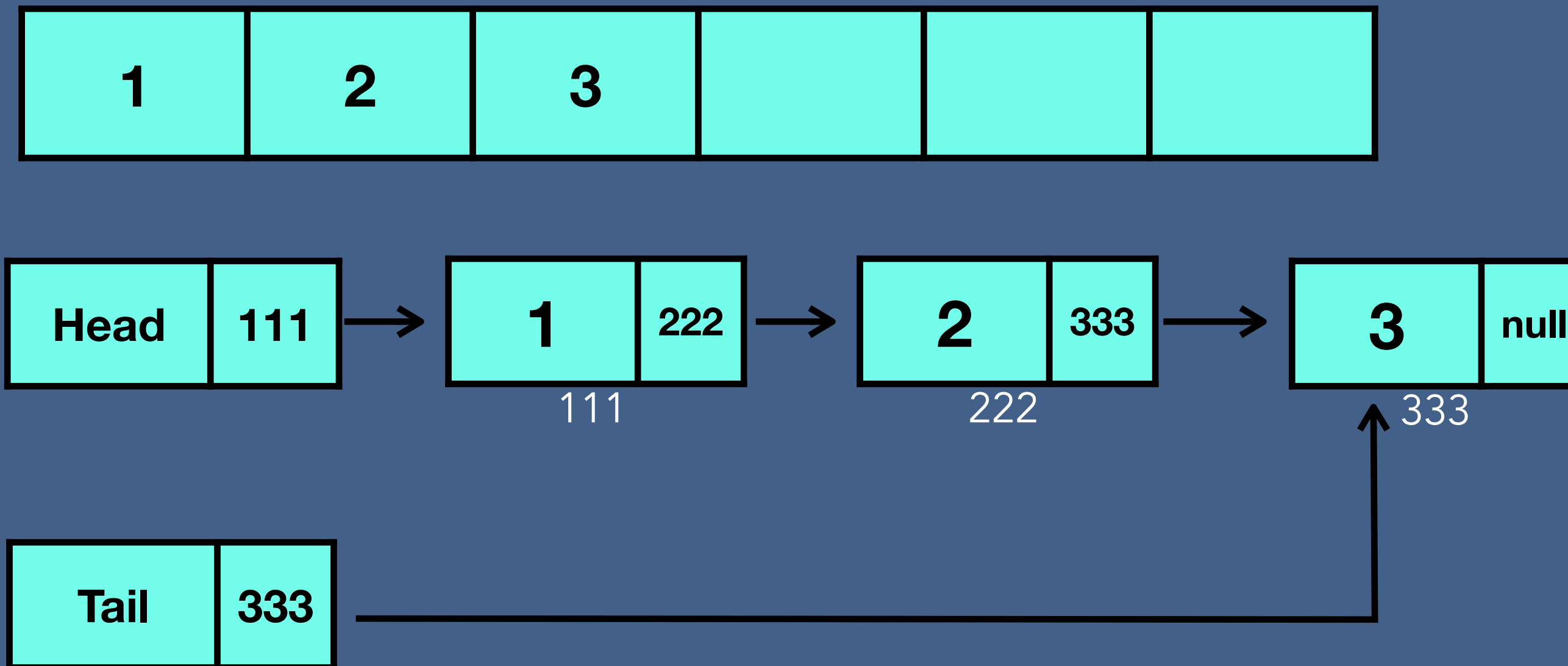
Queue Operations

Implementation

1. Array

- Linear Queue
- Circular Queue

2. Linked List

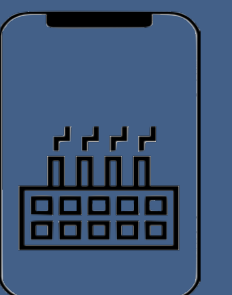


Linear Queue using Array

Create a Queue

```
newQueue = Queue(6)  
beginnigOfQueue = -1  
topOfQueue = -1
```

		[0]	[1]	[2]	[3]	[4]	[5]		

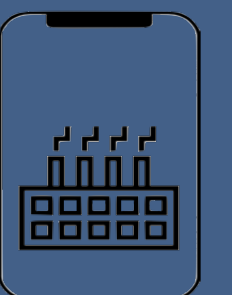


Linear Queue using Array

enqueue Method

```
newQueue.enqueue(1)  
beginnigOfQueue = 0  
topOfQueue = 0
```

		[0]	[1]	[2]	[3]	[4]	[5]		
		1							

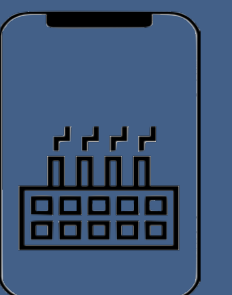


Linear Queue using Array

enqueue Method

```
newQueue.enqueue(2)  
beginnigOfQueue = 0  
topOfQueue = 1
```

		[0]	[1]	[2]	[3]	[4]	[5]		
		1	2						

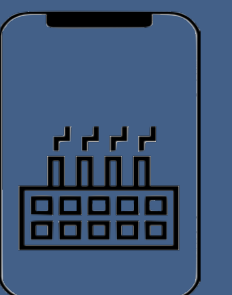


Linear Queue using Array

enqueue Method

```
newQueue.enqueue(3)  
beginnigOfQueue = 0  
topOfQueue = 2
```

		[0]	[1]	[2]	[3]	[4]	[5]		
		1	2	3					



Linear Queue using Array

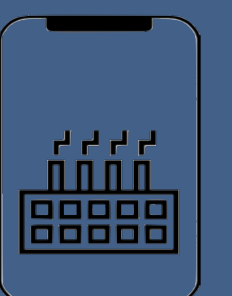
deQueue Method

`newQueue.dequeue()` $\longrightarrow$ 1

`beginningOfQueue = 0`

`topOfQueue = 2`

		[0]	[1]	[2]	[3]	[4]	[5]		
		1	2	3					



Linear Queue using Array

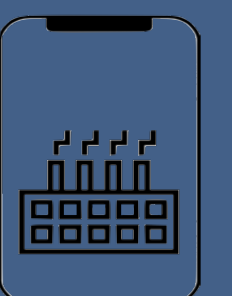
deQueue Method

`newQueue.dequeue()` $\longrightarrow$ 2

`beginningOfQueue = 2`

`topOfQueue = 2`

		[0]	[1]	[2]	[3]	[4]	[5]		
			2	3					



Linear Queue using Array

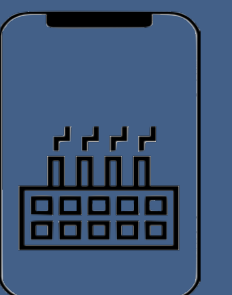
peek Method

`newQueue.peek()` $\longrightarrow$ 1

`beginningOfQueue = 0`

`topOfQueue = 2`

		[0]	[1]	[2]	[3]	[4]	[5]		
		1	2	3					



Linear Queue using Array

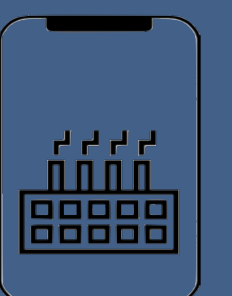
isEmpty Method

`newQueue.isEmpty()` $\longrightarrow$ `False`

`beginningOfQueue = 0`

`topOfQueue = 2`

		[0]	[1]	[2]	[3]	[4]	[5]		
		1	2	3					



Linear Queue using Array

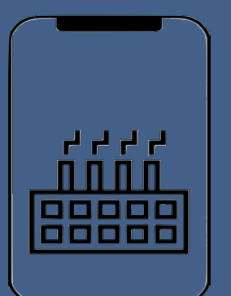
isFull Method

`newQueue.isFull()` $\longrightarrow$ `False`

`beginnigOfQueue = 0`

`topOfQueue = 2`

		[0]	[1]	[2]	[3]	[4]	[5]		
		1	2	3					

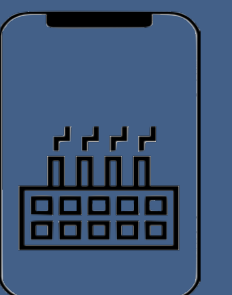


Linear Queue using Array

delete Method

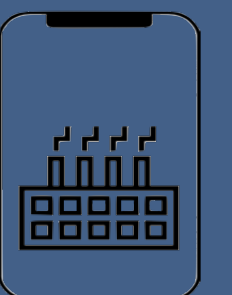
```
newQueue.delete()
```

		[0]	[1]	[2]	[3]	[4]	[5]		
		1	2	3					



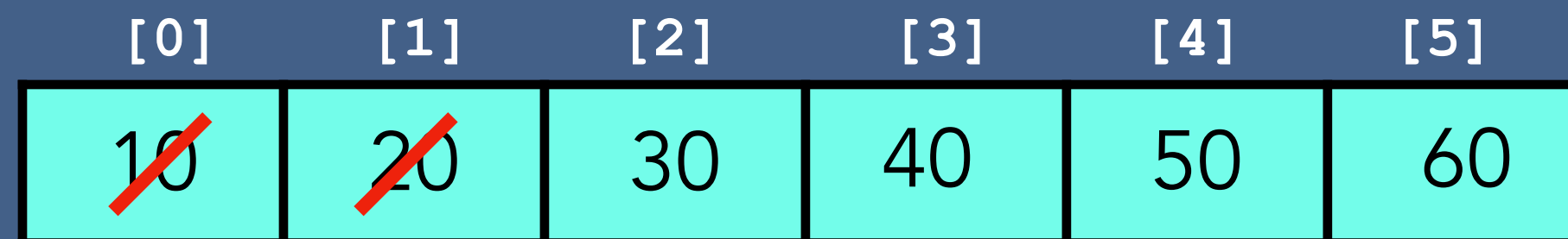
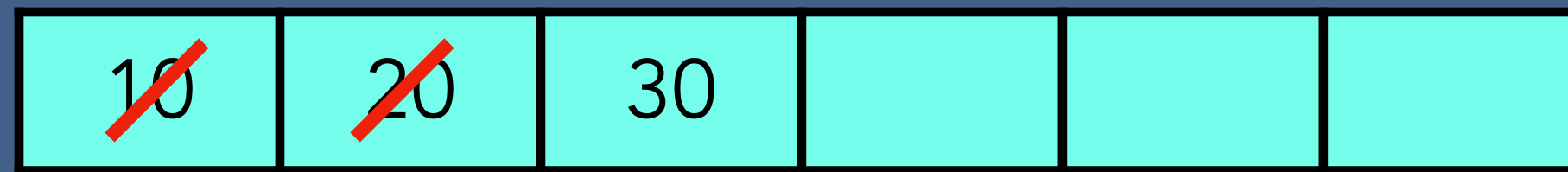
Time and Space Complexity of Linear Queue using Array

Linear Queue - Array	Time complexity	Space complexity
Create Queue	$O(1)$	$O(N)$
enQueue	$O(1)$	$O(1)$
deQueue	$O(1)$	$O(1)$
Peek	$O(1)$	$O(1)$
isEmpty	$O(1)$	$O(1)$
isFull	$O(1)$	$O(1)$
Delete Entire Queue	$O(1)$	$O(1)$

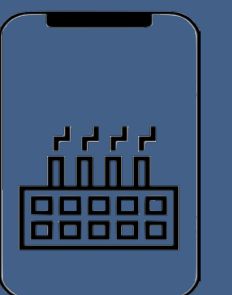


Why do we need Circular Queue?

deQueue method causes blank cells.



$\text{topOfQueue} = 2 + 1 = 3 + 1 = 4 + 1 = 5$

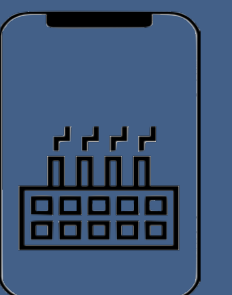


Circular Queue using Array

Create a Queue

```
newQueue = Queue(6)  
beginningOfQueue = -1  
topOfQueue = -1
```

		[0]	[1]	[2]	[3]	[4]	[5]		



Circular Queue using Array

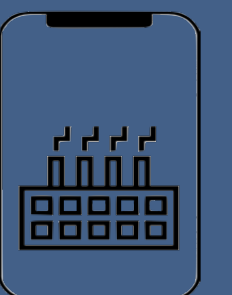
enQueue Method

```
newQueue.enQueue(5)
```

```
beginnigOfQueue = 0
```

```
topOfQueue = 0
```

		[0]	[1]	[2]	[3]	[4]	[5]		
		5							
		↑ ↑							
		B T							



Circular Queue using Array

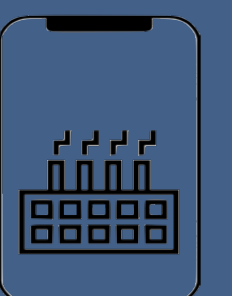
enQueue Method

```
newQueue.enQueue(6)
```

```
beginningOfQueue = 0
```

```
topOfQueue = 0
```

		[0]	[1]	[2]	[3]	[4]	[5]		
		5	6						
		↑ ↑							
		B T							



Circular Queue using Array

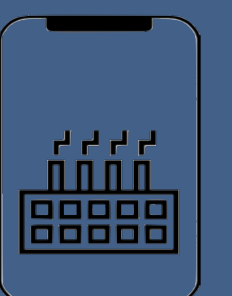
enQueue Method

```
newQueue.enqueue(7)
```

```
beginningOfQueue = 0
```

```
topOfQueue = 2
```

		[0]	[1]	[2]	[3]	[4]	[5]		
		5	6	7					
		↑	↑						
		B	T						



Circular Queue using Array

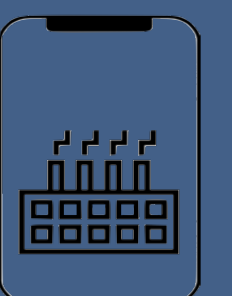
deQueue Method

`newQueue.deQueue()` $\longrightarrow$ 5

`beginningOfQueue = 0`

`topOfQueue = 5`

		[0]	[1]	[2]	[3]	[4]	[5]		
		5	6	7	8	9	10		
		↑					↑		
		B					T		



Circular Queue using Array

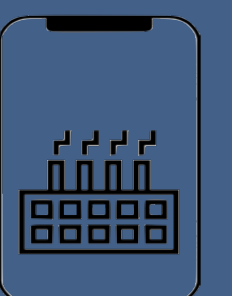
deQueue Method

`newQueue.deQueue()` $\longrightarrow$ 6

`beginningOfQueue = 2`

`topOfQueue = 5`

		[0]	[1]	[2]	[3]	[4]	[5]		
		5	6	7	8	9	10		
			↑				↑		
			B				T		



Circular Queue using Array

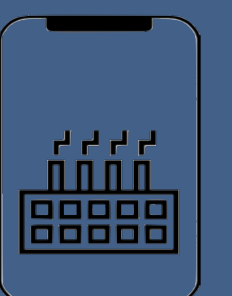
deQueue Method

```
newQueue.enqueue(1)
```

```
beginningOfQueue = 2
```

```
topOfQueue = 5
```

		[0]	[1]	[2]	[3]	[4]	[5]		
		1	6	7	8	9	10		
				↑			↑		
				B			T		



Circular Queue using Array

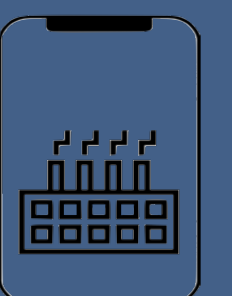
Peek Method

`newQueue.peek()` $\longrightarrow$ 6

`beginningOfQueue = 1`

`topOfQueue = 5`

		[0]	[1]	[2]	[3]	[4]	[5]		
			6	7	8	9	10		
			↑				↑		
			B				T		



Circular Queue using Array

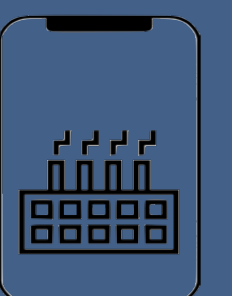
isFull Method

`newQueue.isFull()` $\longrightarrow$ `False`

`beginningOfQueue = 1`

`topOfQueue = 5`

		[0]	[1]	[2]	[3]	[4]	[5]		
			6	7	8	9	10		
			↑				↑		
			B				T		



Circular Queue using Array

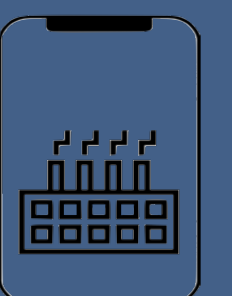
isFull Method

`newQueue.isFull()` $\longrightarrow$ `True`

`beginningOfQueue = 0`

`topOfQueue = 5`

		[0]	[1]	[2]	[3]	[4]	[5]		
		5	6	7	8	9	10		
		↑					↑		
		B					T		



Circular Queue using Array

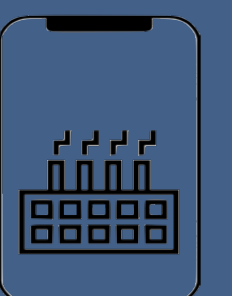
isFull Method

`newQueue.isFull()` $\longrightarrow$ `True`

`beginningOfQueue = 1`

`topOfQueue = 0`

		[0]	[1]	[2]	[3]	[4]	[5]		
		5	6	7	8	9	10		
		↑	↑						
		T	B						



Circular Queue using Array

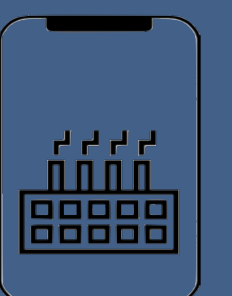
isEmpty Method

`newQueue.isEmpty()` $\longrightarrow$ `False`

`beginningOfQueue = 1`

`topOfQueue = 0`

		[0]	[1]	[2]	[3]	[4]	[5]		
		5	6	7	8	9	10		
		↑	↑						
		T	B						

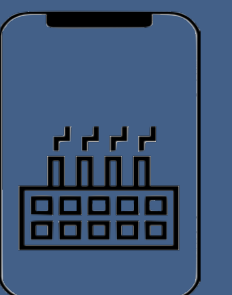


Circular Queue using Array

Delete Method

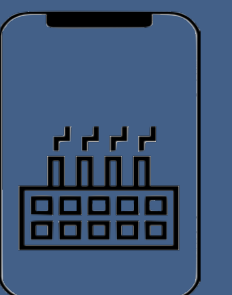
```
newQueue.delete()  
beginnigOfQueue = 1  
topOfQueue = 0
```

		[0]	[1]	[2]	[3]	[4]	[5]		
		5	6	7	8	9	10		
		↑	↑						
		T	B						



Time and Space Complexity of Circular Queue using Array

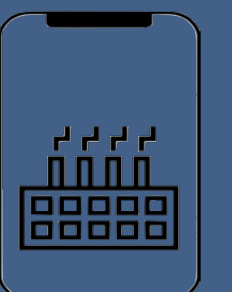
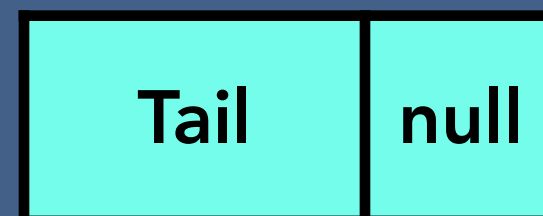
Circular Queue - Array	Time complexity	Space complexity
Create Queue	$O(1)$	$O(N)$
enQueue	$O(1)$	$O(1)$
deQueue	$O(1)$	$O(1)$
Peek	$O(1)$	$O(1)$
isEmpty	$O(1)$	$O(1)$
isFull	$O(1)$	$O(1)$
Delete Entire Queue	$O(1)$	$O(1)$



Queue using Linked List

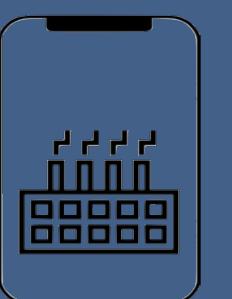
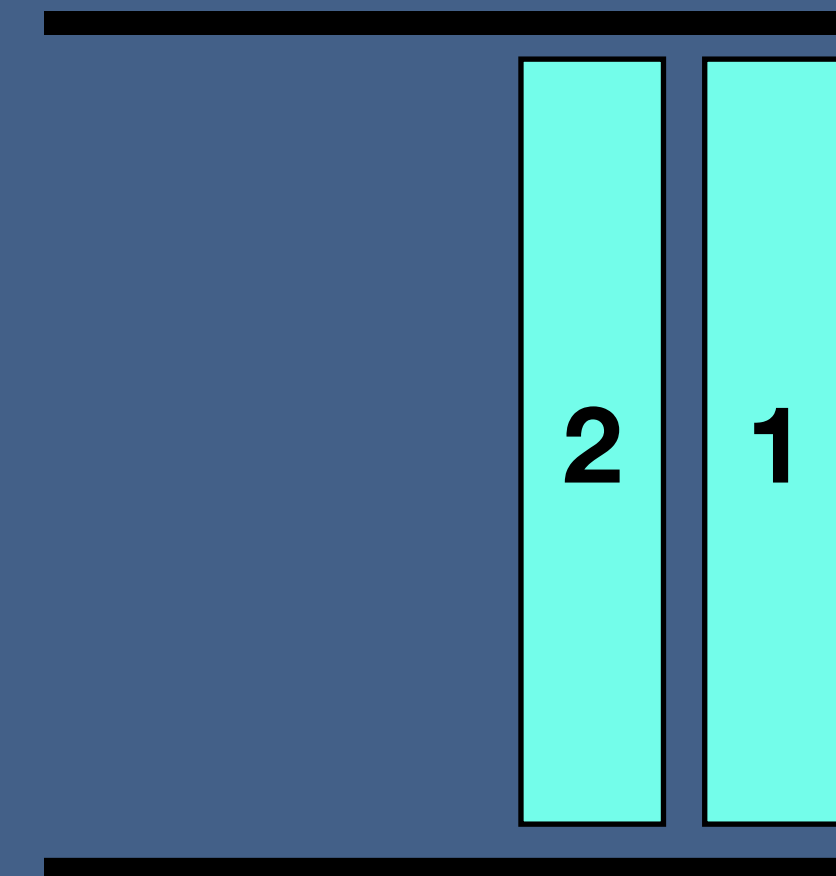
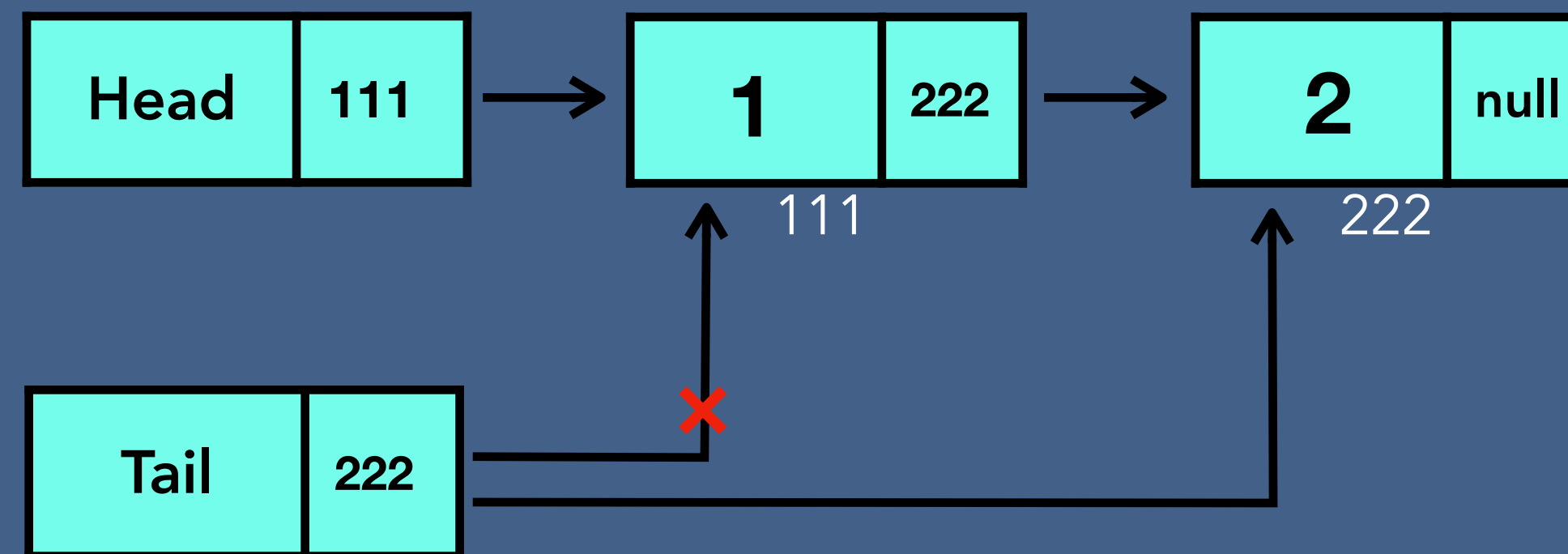
Create a Queue

Create an object of Linked List class



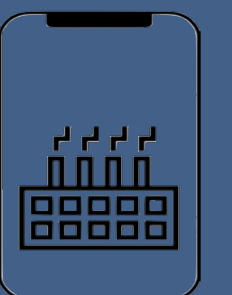
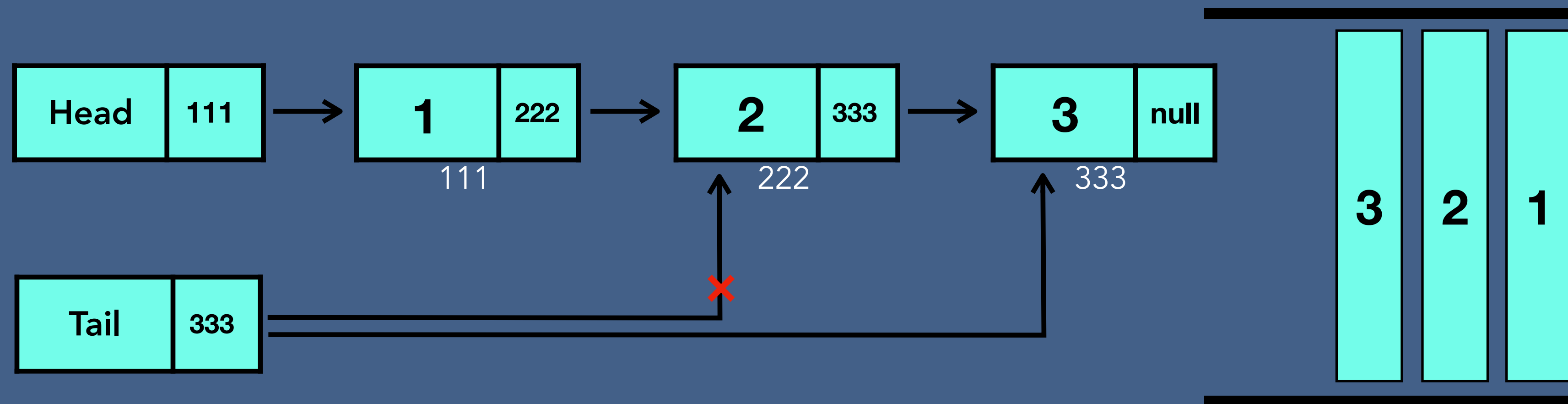
Queue using Linked List

enqueue() Method



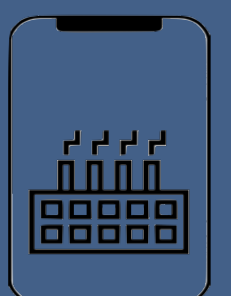
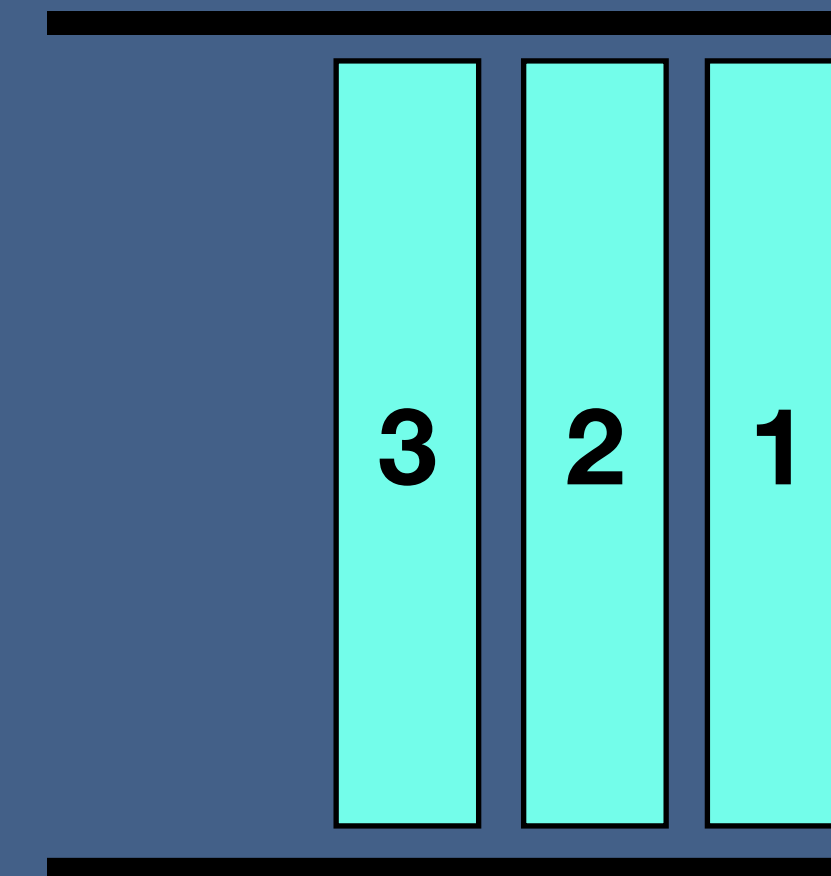
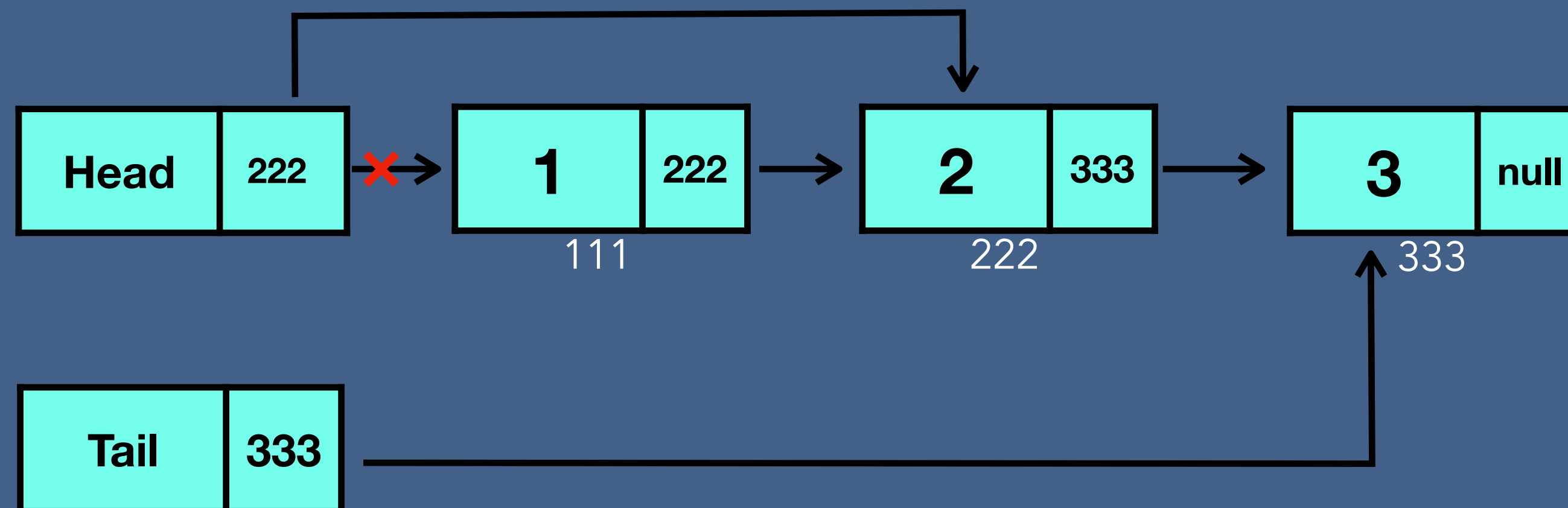
Queue using Linked List

enqueue() Method



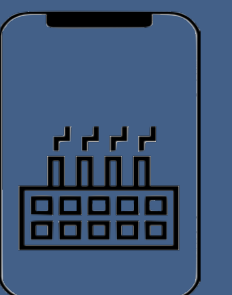
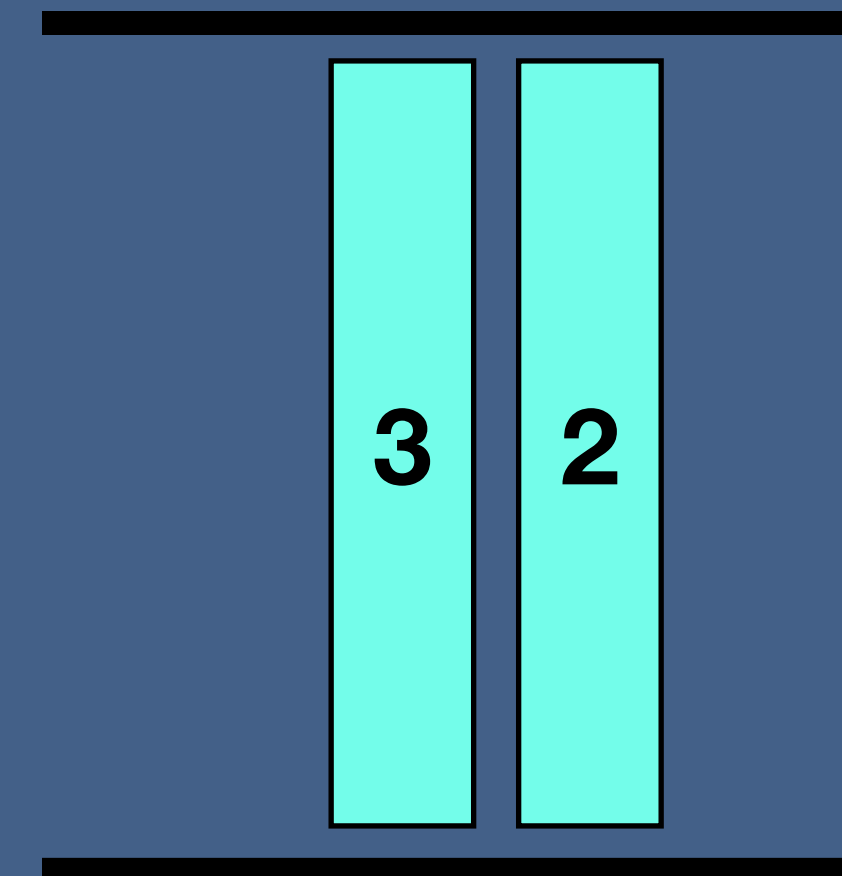
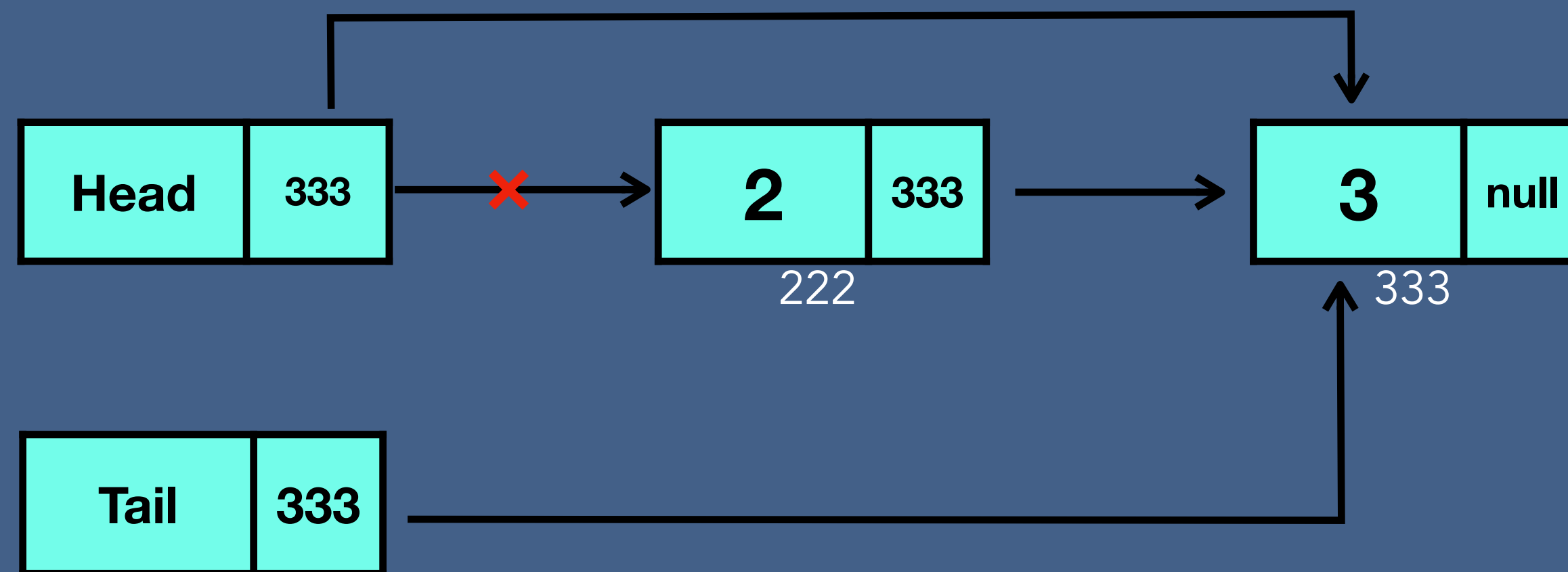
Queue using Linked List

deQueue() Method



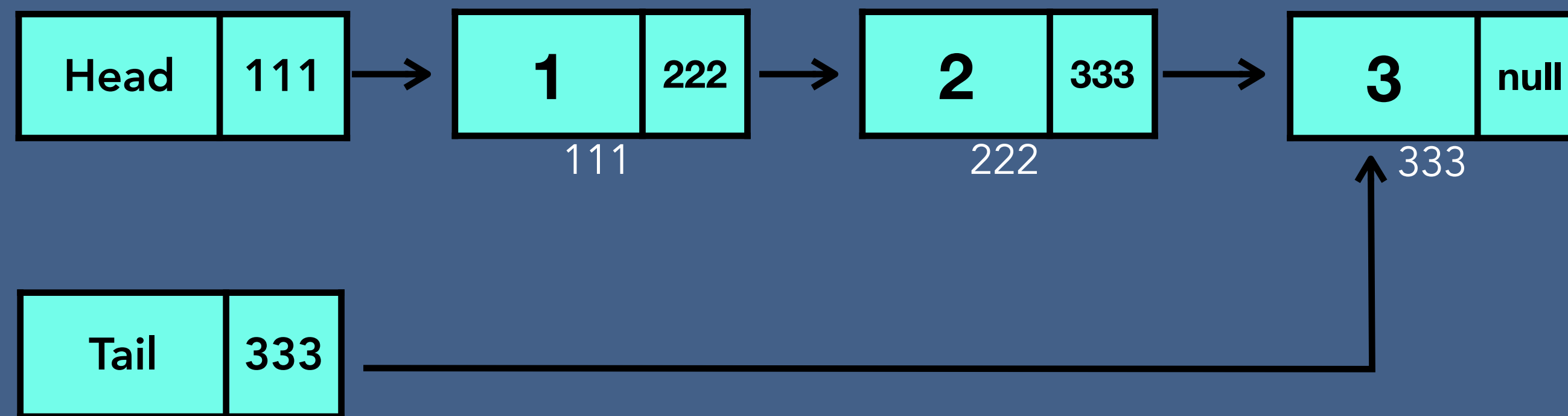
Queue using Linked List

deQueue() Method

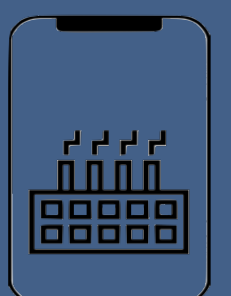
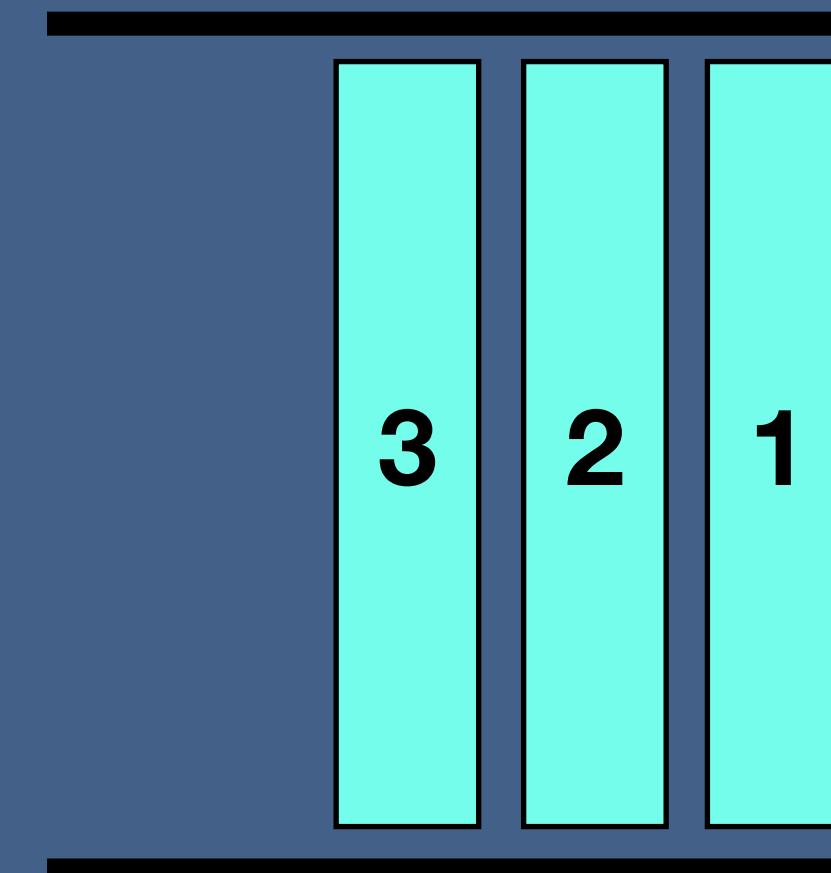


Queue using Linked List

peek() Method

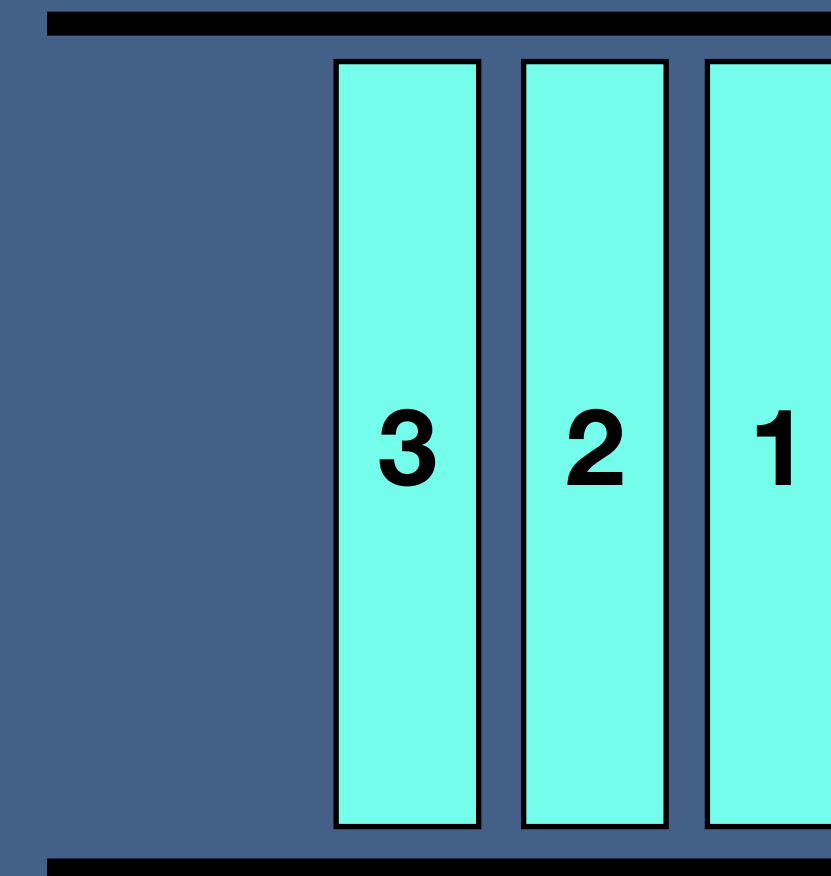
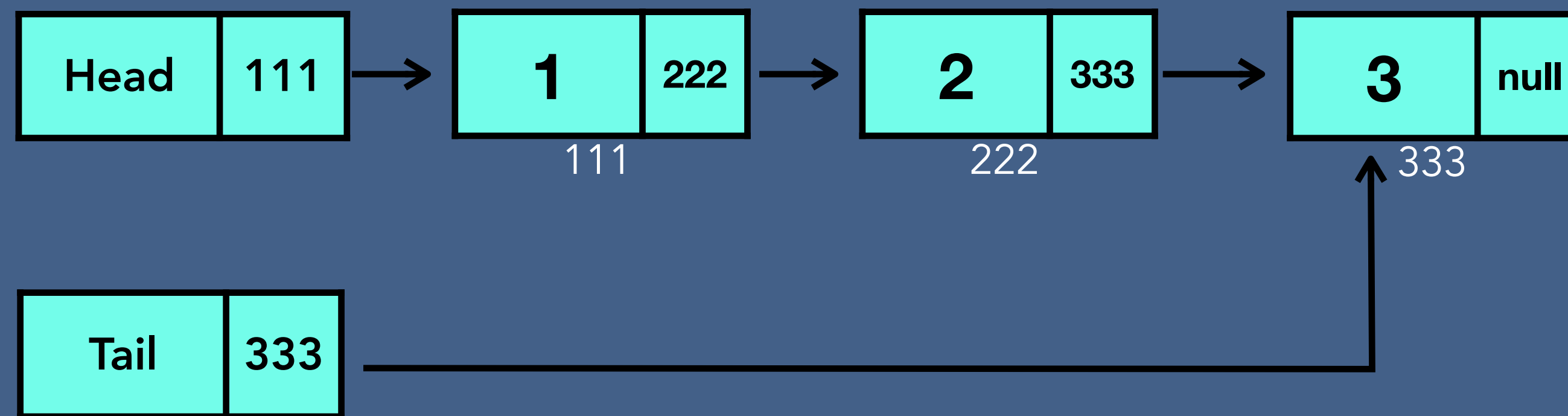


peek()
return head.value

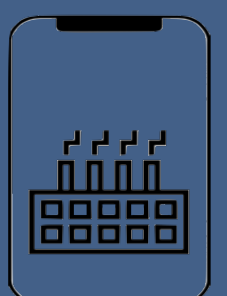


Queue using Linked List

isEmpty() Method

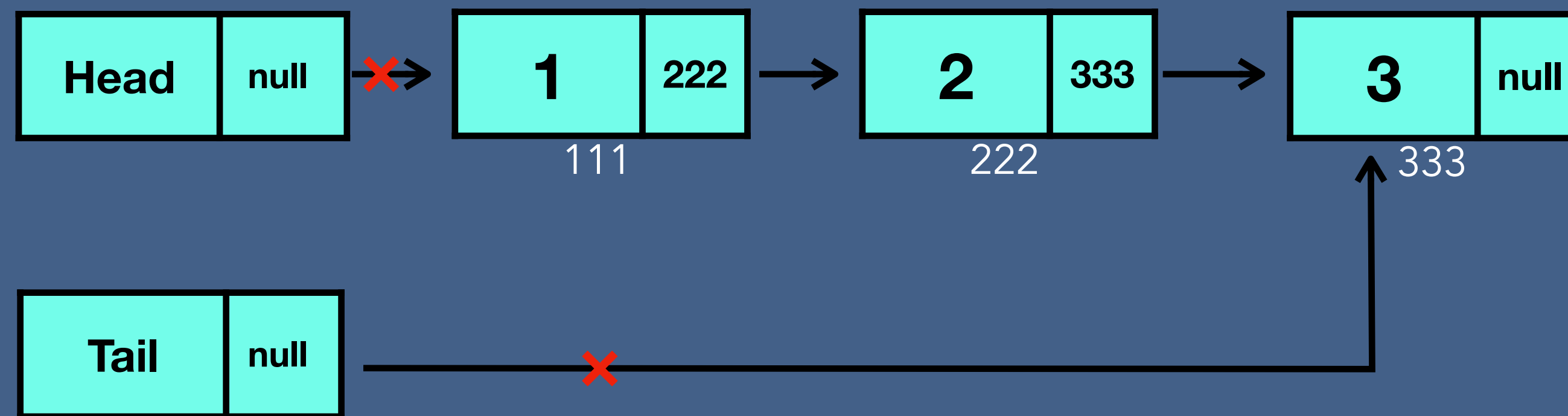


isEmpty()
If head is Null:
True

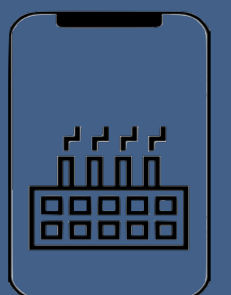
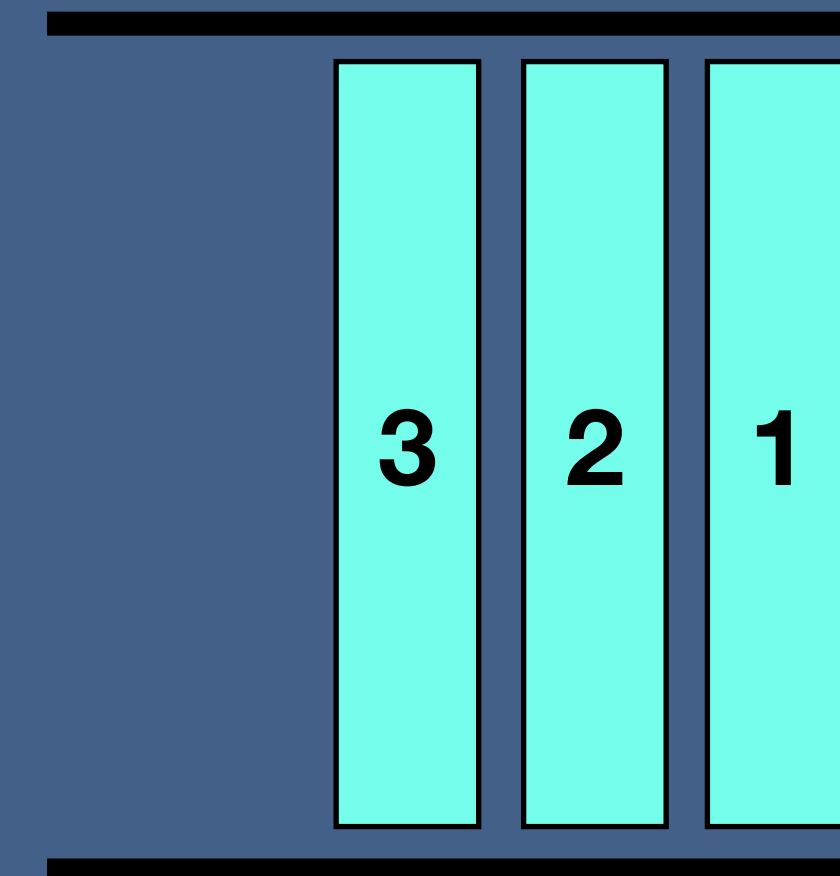


Queue using Linked List

delete() Method

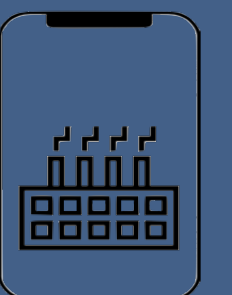


```
delete()
head = Null
tail = Null
```



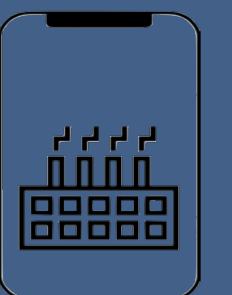
Time and Space Complexity of Queue using Linked List

Queue - Linked List	Time complexity	Space complexity
Create Queue	$O(1)$	$O(1)$
enQueue	$O(1)$	$O(1)$
deQueue	$O(1)$	$O(1)$
Peek	$O(1)$	$O(1)$
isEmpty	$O(1)$	$O(1)$
Delete Entire Queue	$O(1)$	$O(1)$



Queue - Array vs Linked List

	Array		Linked List	
	Time complexity	Space complexity	Time complexity	Space complexity
Create Queue	O(1)	O(n)	O(1)	O(1)
Enqueue	O(1)	O(1)	O(1)	O(1)
Dequeue	O(1)	O(1)	O(1)	O(1)
Peek	O(1)	O(1)	O(1)	O(1)
isEmpty	O(1)	O(1)	O(1)	O(1)
isFull	O(1)	O(1)	-	-
Delete Entire Queue	O(1)	O(1)	O(1)	O(1)
Space efficient?		NO		Yes



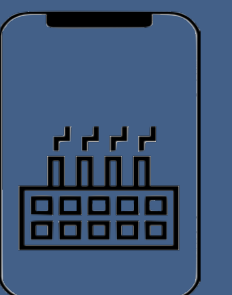
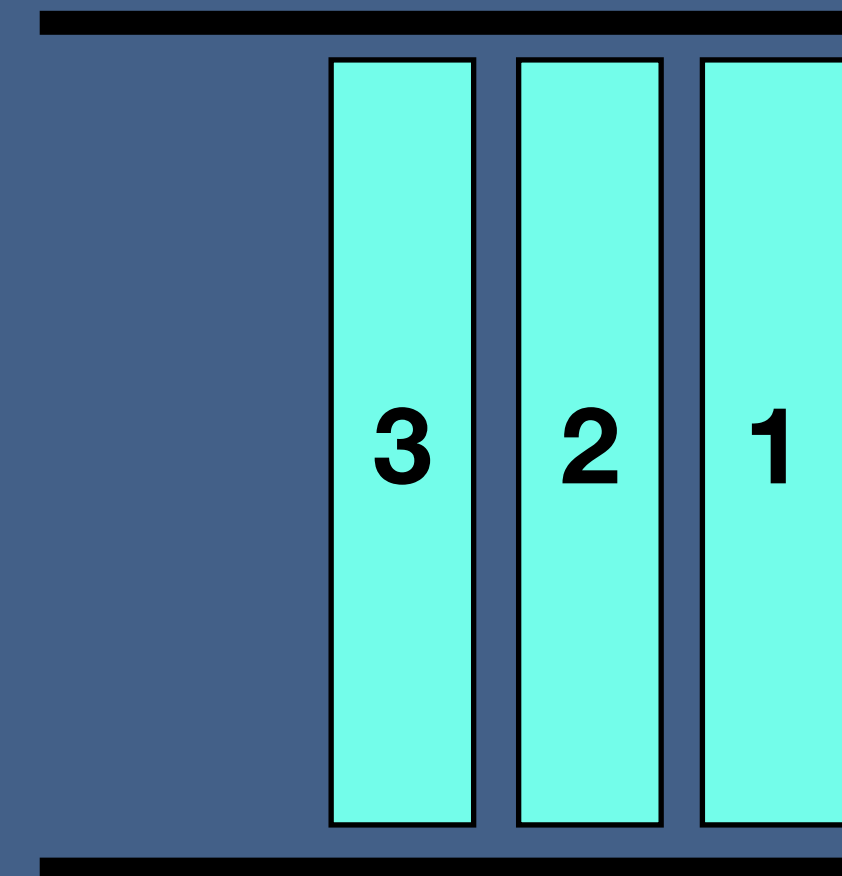
When to Use/Avoid Queue?

Use:

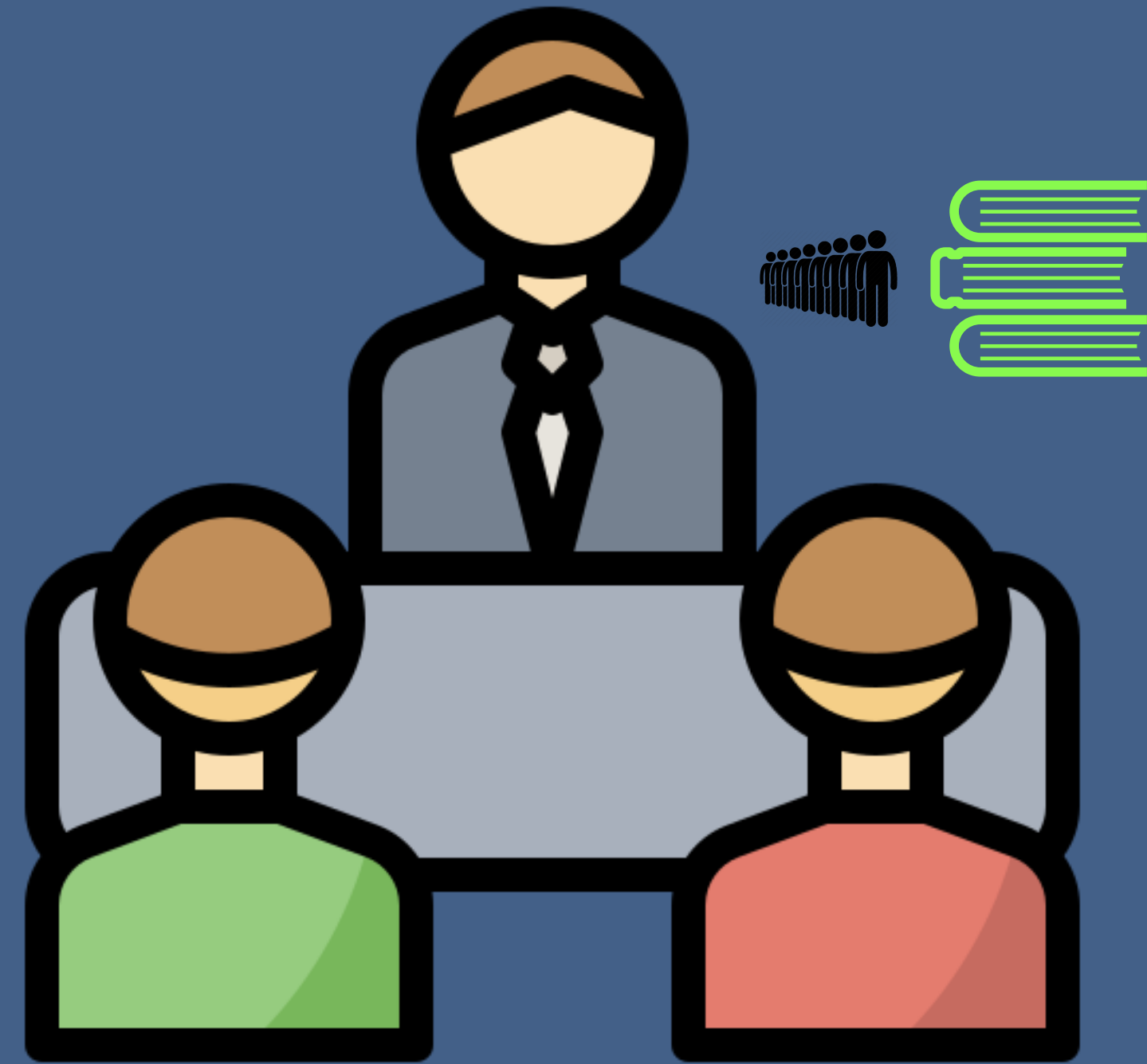
- FIFO functionality
- The chance of data corruption is minimum

Avoid:

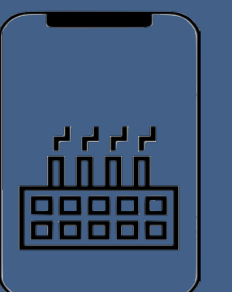
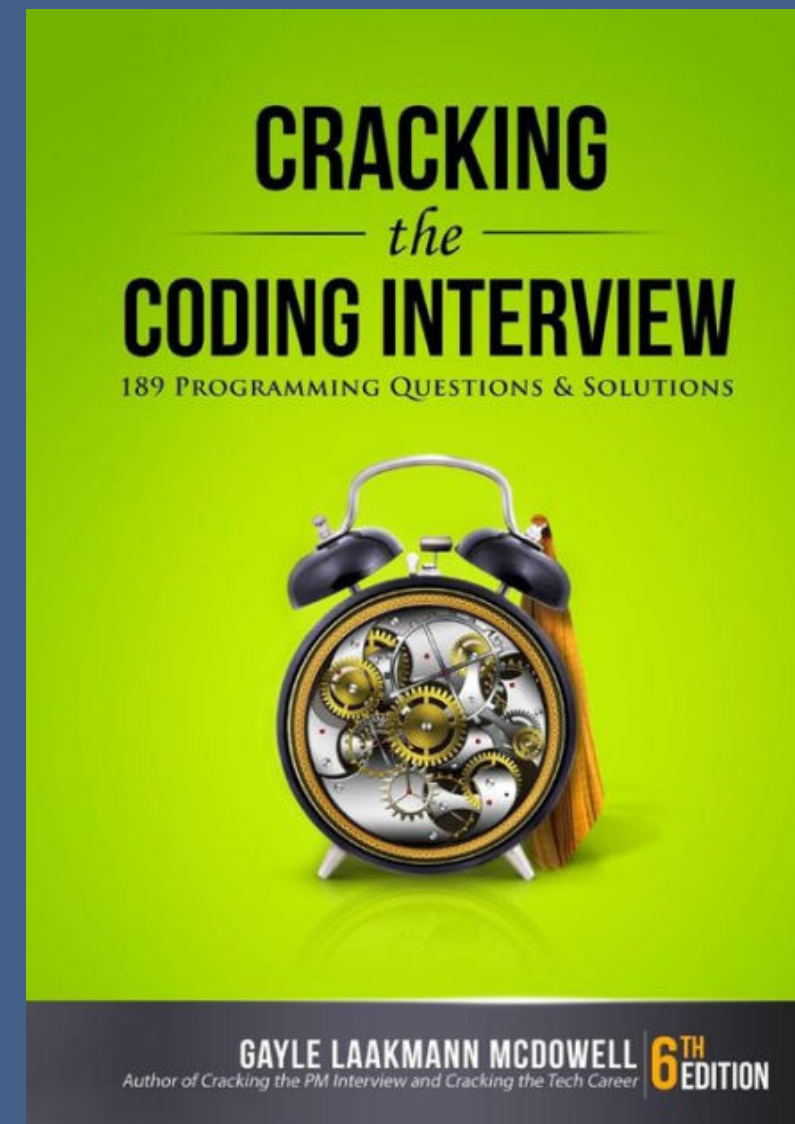
- Random access is not possible



Stack and Queue Interview Questions

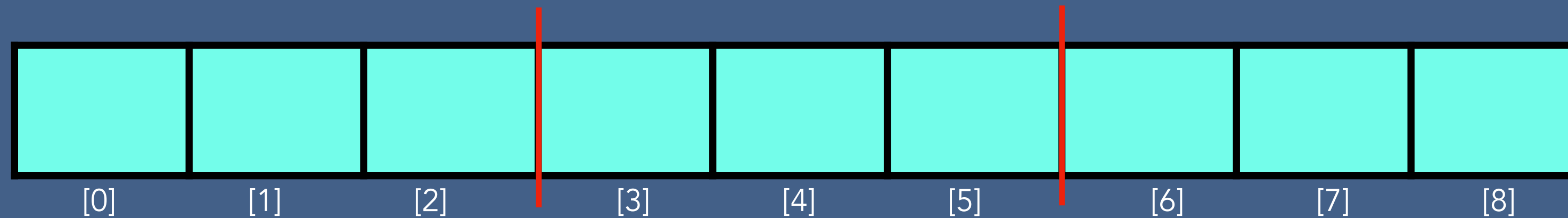


Stack and Queue Interview Questions

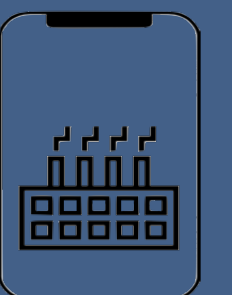


Three in One

Describe how you could use a single Array to implement three stacks.



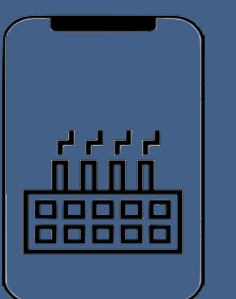
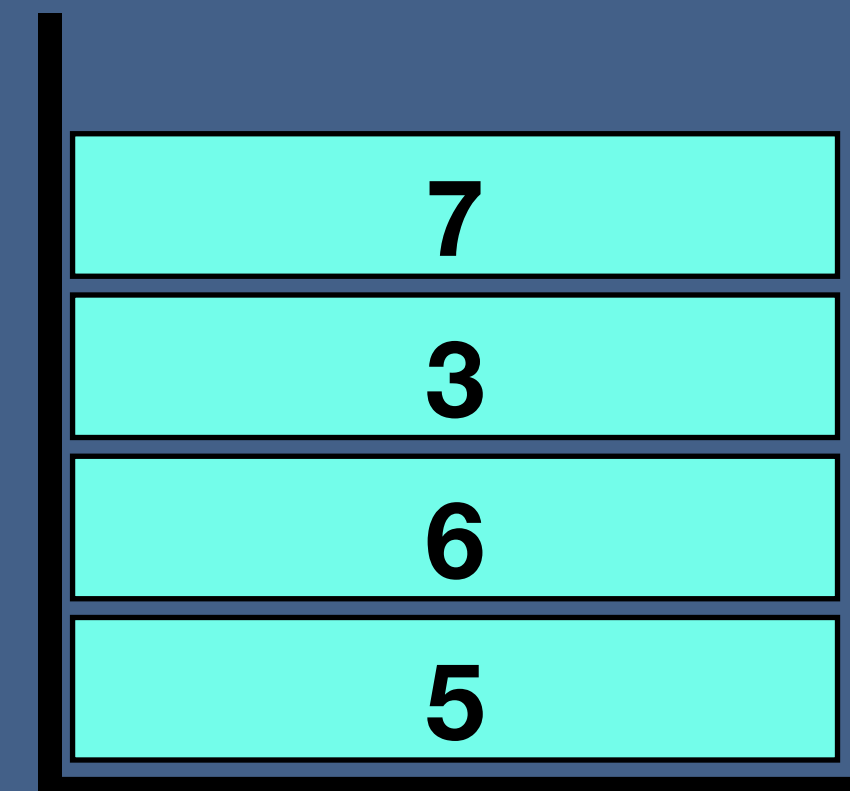
For Stack 1 — [0], [1], [2]	→	$[0, n/3)$
For Stack 2 — [3], [4], [5]	→	$[n/3, 2n/3)$
For Stack 3 — [6], [7], [8]	→	$[2n/3, n)$



Stack Min

How would you design a stack which, in addition to push and pop, has a function min which returns the minimum element? Push, pop and min should all operate in $O(1)$.

push(5)	min() → 5
push(6)	min() → 5
push(3)	min() → 3
push(7)	min() → 3
pop()	min() → 3
pop()	min() → 5

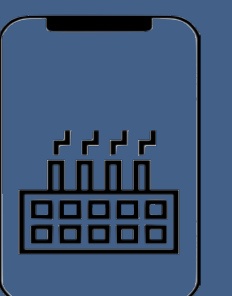
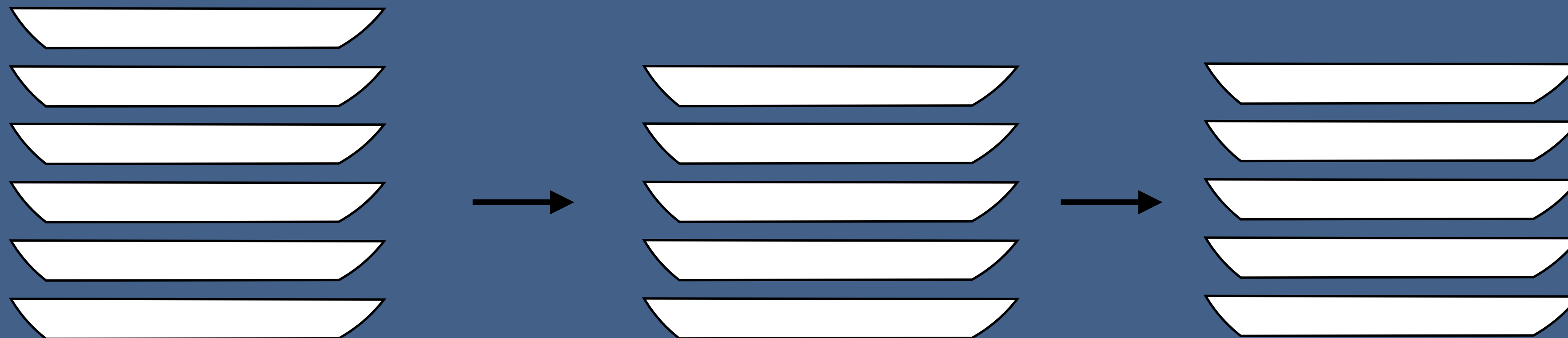


Stack of Plates

Imagine a (literal) stack of plates. If the stack gets too high, it might topple. Therefore, in real life, we would likely start a new stack when the previous stack exceeds some threshold. Implement a data structure `SetOfStacks` that mimics this. `SetOfStacks` should be composed of several stacks and should create a new stack once the previous one exceeds capacity, `SetOfStacks.push()` and `SetOfStacks.pop()` should behave identically to a single stack (that is, `pop()` should return the same values as it would if there were just a single stack).

Follow Up:

Implement a function `popAt (int index)` which performs a pop operation on a specific sub - stack.



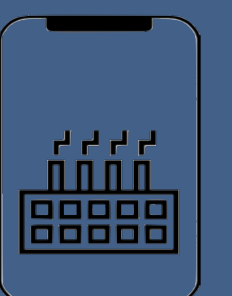
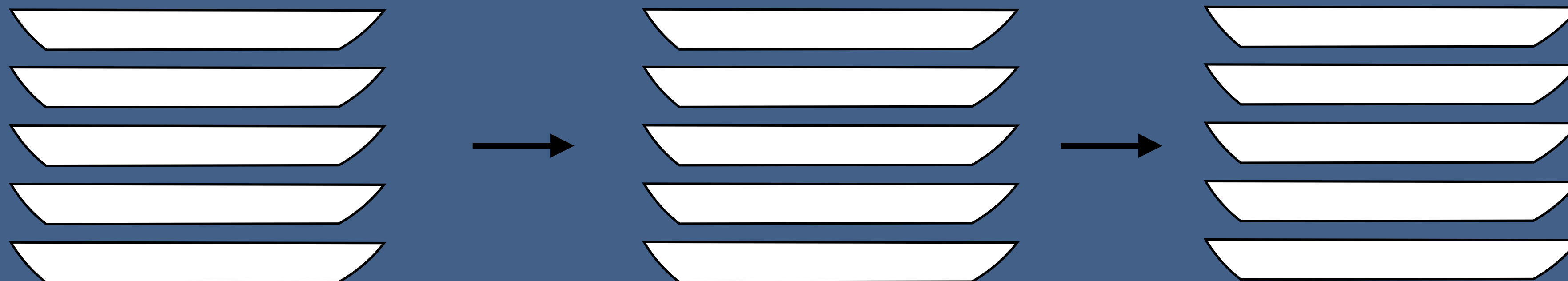
Stack of Plates

Imagine a (literal) stack of plates. If the stack gets too high, it might topple. Therefore, in real life, we would likely start a new stack when the previous stack exceeds some threshold. Implement a data structure `SetOfStacks` that mimics this. `SetOfStacks` should be composed of several stacks and should create a new stack once the previous one exceeds capacity, `SetOfStacks.push()` and `SetOfStacks.pop()` should behave identically to a single stack (that is, `pop()` should return the same values as it would if there were just a single stack).

Follow Up:

Implement a function `popAt (int index)` which performs a pop operation on a specific sub - stack.

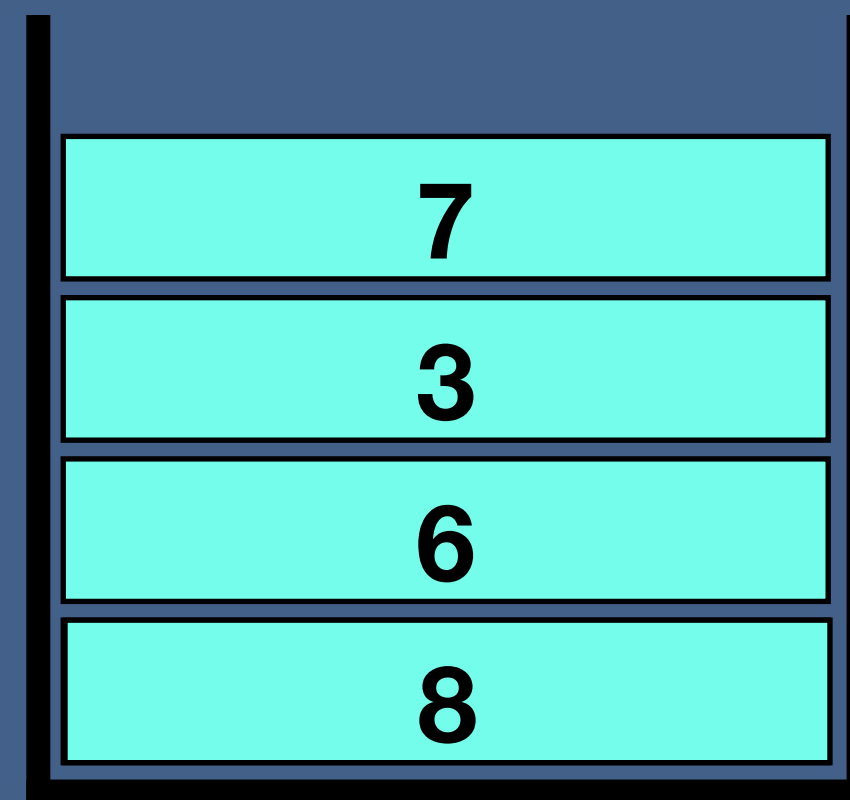
`pop()`



Queue via Stacks

Implement Queue class which implements a queue using two stacks.

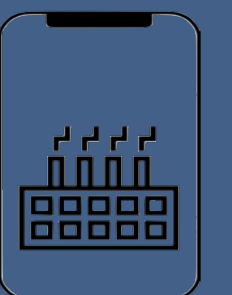
~~Dequeue()~~



Stack1



Stack2



Animal Shelter

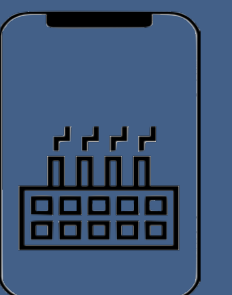
An animal shelter, which holds only dogs and cats, operates on a strictly "first in, first out" basis. People must adopt either the "oldest" (based on arrival time) of all animals at the shelter, or they can select whether they would prefer a dog or a cat (and will receive the oldest animal of that type). They cannot select which specific animal they would like. Create the data structures to maintain this system and implement operations such as enqueue, dequeueAny, dequeueDog, and dequeueCat.

First In First Out



Enqueue(Dog)

Enqueue(Cat)



Animal Shelter

An animal shelter, which holds only dogs and cats, operates on a strictly "first in, first out" basis. People must adopt either the "oldest" (based on arrival time) of all animals at the shelter, or they can select whether they would prefer a dog or a cat (and will receive the oldest animal of that type). They cannot select which specific animal they would like. Create the data structures to maintain this system and implement operations such as enqueue, dequeueAny, dequeueDog, and dequeueCat.

First In First Out



DequeueAny()

DequeueDog()

DequeueCat()

